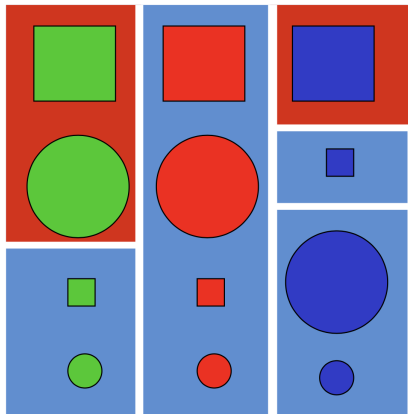
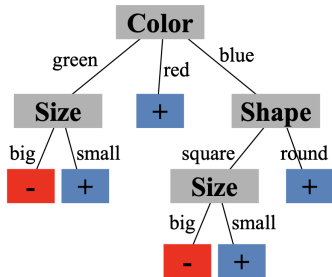


Decision Trees

Toy Example



Possibly interesting idea but how could we transfer the information contained in such a tree to be processable by a machine?

Feature Representation

Extract **attributes** or **features** (e.g., color, size, shape) with machine readable values (e.g., continuous, discrete) from real world 'objects'.

Attributes describe characteristic traits of the objects so that different objects have different attribute values; objects with the same values are indistinguishable from one another.

Attributes for an object are usually collected in a vector $x \in \mathbb{R}^D$, so that D is the number of attributes and the value of the d -th attribute is encoded in dimension x_d of $x = (x_1, \dots, x_D)^\top$.

Since there are many objects, we address them by numbers, e.g., for N objects, we have attribute vectors $x_1, \dots, x_N$, with $x_n = (x_{n1}, \dots, x_{nD})^\top$.

Open Question

What would be an appropriate feature representation for

- text documents
- emails
- tweets
- pictures
- sensor readings
- click logs
- ...

?

Decision Trees

Given training data $\{(x_n, y_n)\}_{n=1, \dots, N}$ with $x_n \in \mathbb{R}^D$ and $y_n \in Y$ where Y is a set of discrete class labels (e.g., $Y = \{spam, ham\}$)

A decision tree implements a mapping $f : X \rightarrow Y$ as follows:

- Each internal node tests an attribute (dimension) X_d
- There is one branch for each possible value of X_d
- Every leaf node is associated with a class label $y \in Y$
- A new input x^{new} is classified by traversing the tree from root to leaf and output the corresponding label of that leaf

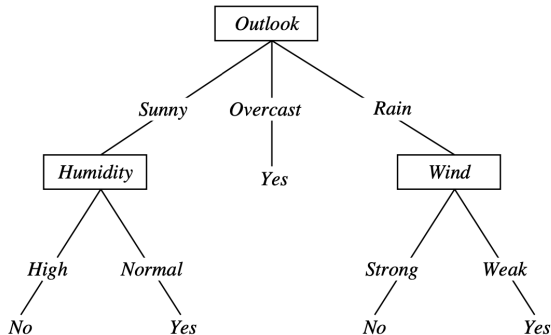
Example

$\langle x_i, y_i \rangle$

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

- Columns denote features X_d
- Rows denote labeled instances (x_n, y_n)
- Class label denotes whether a tennis game was played, hence $y_n \in \{\text{Yes}, \text{No}\}$

Decision Trees & Rules



The decision tree implements the following set of rules:

- IF (*outlook* = *sunny* $\wedge$ *humidity* = *normal*) THEN *y* = *Yes*
- IF (*outlook* = *overcast*) THEN *y* = *yes*
- IF (*outlook* = *rain* $\wedge$ *wind* = *weak*) THEN *y* = *Yes*
- ...

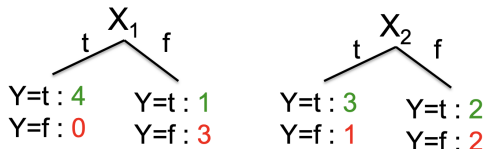
Human interpretable!

How to find a good tree?

- Learning the simplest (smallest) decision tree is NP-complete (Hyafil & Rivest, 1976)
- Idea: A greedy approach
 - Start from empty decision tree (i.e. root)
 - Split on next best attribute (feature)
 - Recurse until termination criterion fulfilled

Splitting: How to choose a good attribute?

Would we prefer to split on attribute X_1 or X_2 ?



Idea: Use counts at leaves to define probability distributions and measure uncertainty!

X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F
F	T	F
F	F	F

Measuring Uncertainty

A good split decreases uncertainty about classification after the split

- point distributions (all true or all false) realize the perfect split
- uniform distributions the worst split
- What about distributions in between?

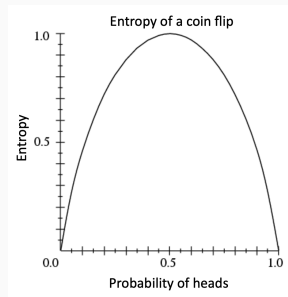
Entropy

The **entropy** $H(Y)$ of a random variable Y is given by

$$H(Y) = - \sum_{r=1}^R P(Y = y_r) \log_2 P(Y = y_r)$$

More uncertainty, more entropy!

Information Theory interpretation:
 $H(Y)$ is the expected number of bits
needed to encode a randomly drawn
value of Y (under most efficient
code)



Entropy Example

$$H(Y) = - \sum_{r=1}^R P(Y = y_r) \log_2 P(Y = y_r)$$

$$P(Y = \text{True}) = \frac{5}{6}$$

$$P(Y = \text{False}) = \frac{1}{6}$$

$$H(Y) = -\frac{5}{6} \log_2 \frac{5}{6} - \frac{1}{6} \log_2 \frac{1}{6} \approx 0.65$$

X ₁	X ₂	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F

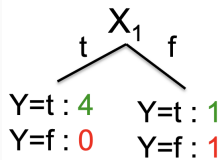
Conditional Entropy

The **conditional entropy** $H(Y|X)$ of a random variable Y conditioned on random variable X is given by

$$H(Y|X) = - \sum_{j=1}^J P(X = x_j) \sum_{r=1}^R P(Y = y_r | X = x_j) \log_2 P(Y = y_r | X = x_j)$$

Example:

$$P(X_1 = \text{True}) = \frac{4}{6}$$
$$P(X_1 = \text{False}) = \frac{2}{6}$$



X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F

$$H(Y|X_1) = -\frac{4}{6} (1 \log_2 1 + 0 \log_2 0) - \frac{2}{6} \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = \frac{2}{6}$$

Information Gain

Decrease in entropy (uncertainty) after splitting

$$IG(X) = H(Y) - H(Y|X)$$

For our running example, we obtain

$$IG(X_1) = H(Y) - H(Y|X_1) = 0.65 - 0.33 > 0$$

Thus, $IG(X_1) > 0$ and the split contributes positively to further decrease uncertainty

X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F

Learning Decision Trees – Algorithm Blueprint

Given a training sample $\{(x_n, y_n)\}_{n=1, \dots, N}$ with $x_n \in \mathbb{R}^D, y_n \in Y$

- Start from empty decision tree
- Split on next best attribute (feature)
 - Use, for example, information gain to select attribute via

$$\operatorname{argmax}_{1 \leq d \leq D} IG(X_d) = \operatorname{argmax}_d H(Y) - H(Y|X_d)$$

- Recurse

In practice, training samples are often converted into matrices X and Y as follows:

Given $\{(x_n, y_n)\}_{n=1, \dots, N}$ with $x_n \in \mathbb{R}^D$, $y_n \in Y$, stack input vectors x_n into a matrix X so that the n -th row contains the n -th instance, i.e.,

$$X = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{pmatrix}$$

Matrix X is called **design matrix**. Similarly, target variables y_n are stacked up to form a **label vector** Y given by

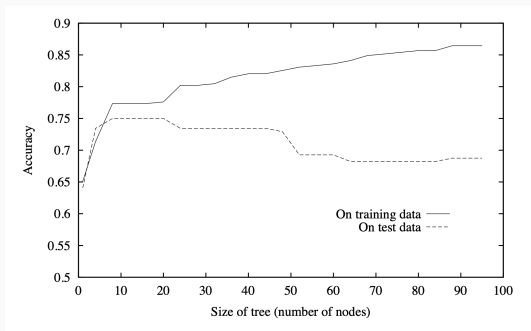
$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix}$$

ID3 Algorithm

ID3(Design matrix X , label vector Y)

- Create a *Root* for the tree
- If $\text{Var}(Y) = 0$, return leaf node with label $y \in Y$
- If $\text{dim}(X, 1) = 0$ or $\text{dim}(X, 2) = 0$, return leaf node with majority label in Y
- $X_d = \text{argmax}_{1 \leq d \leq D} IG(X_d) = \text{argmax}_d H(Y) - H(Y|X_d)$
- $\text{Root} \leftarrow X_d$
- For each possible value $r = 1, \dots, R$ of X_d
 - Add new branch below Root with $X_d = r$
 - Let $I = \{i : X_{i,d} = r\}$
 - If $I = \emptyset$
 - Then add leaf node with majority label $y_{\text{majority}} \in Y$
 - Else add ID3($\{X_{[I,k]} : k \neq d\}$, $Y_{[I]}$)
- Return Root

Overfitting

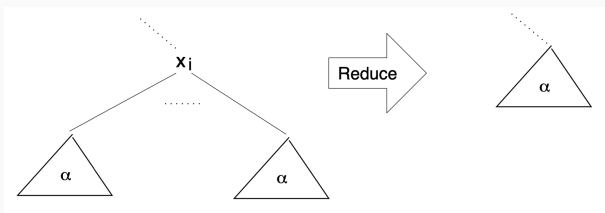


- Given a hypothesis space $\mathcal{H}$, a hypothesis $h \in \mathcal{H}$ is said to **overfit** the training data if there exists some alternative hypothesis $h' \in \mathcal{H}$ such that h has smaller error than h' over the training, but h' has smaller error than h over the entire distribution of instances.

- Reasons for overfitting:
 - Noise in the data
 - Number training examples is too small to produce a representative sample of the target function
- How to avoid overfitting:
 - Stop growing the tree before it reaches the point where it classifies the training data w/out error
 - Allow overfitting and then post-prune the tree (often more successful in practice!)
 - Collect more data
- How to determine the perfect tree size:
 - Maintain a separate validation set to evaluate utility of post-pruning
 - Apply statistical tests to estimate whether expanding (or pruning) improves the current hypothesis

Pruning (example)

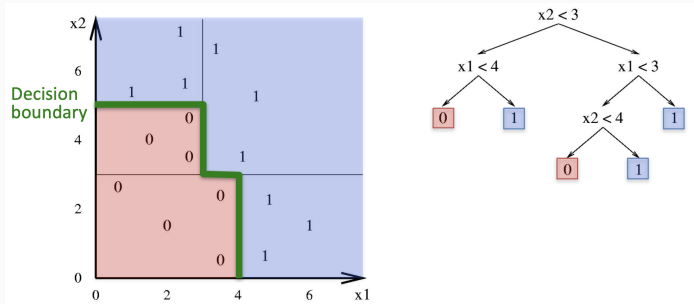
- Every node is considered a candidate for pruning



- **Pruning** a decision node consists of removing the subtree rooted at the node, making it a leaf node and assigning the most common classification of the training examples affiliated with that node
- Nodes are removed only if the resulting tree performs not worse than the original tree over the validation set
- Pruning starts with the node whose removal most increases accuracy and continues until further pruning is harmful

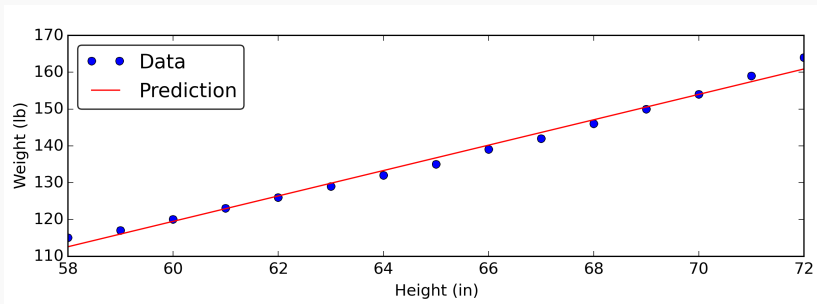
Decision Boundaries

- Decision trees divide the feature space into axis-parallel (hyper-)rectangles
- Each rectangular region is labeled with one label (or a probability distribution over labels)
- This leads to piece-wise linear decision hyperplanes



Motivating Linear Regression

Linear Regression



Setting:

- Labeled data set $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, $N \in \mathbb{N}$
- With continuous input and output space
- Input data points $x_n \in \mathcal{X} = \mathbb{R}^D$
- Every data point x_n has a label $y_n \in \mathcal{Y} = \mathbb{R}$

Supervised Machine Learning

A step back: supervised machine learning

Algorithm that **learns**
to perform a **task**
from **experience**

- **Goal:** mapping $f_w : \underbrace{\mathcal{X}}_{\text{Inputs}} \rightarrow \underbrace{\mathcal{Y}}_{\text{Outputs}}$
- Algorithm **learns** parameter set w
- To perform **task** f
- From **experience** $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$
- Typical assumption: $f_w(x) = \underbrace{w^\top x}_{\text{linear model}}$

Probabilistic approach

- Framing: Find a (deterministic) mapping from random variable X (input) to random variable Y (target).
- The relation between X and Y is given by a (generally unknown) joint probability density function $p_{X,Y}(x,y)$.
- If $p_{X,Y}(x,y)$ was known, we could use

$$p_{Y|X}(y|x) = \frac{p_{X,Y}(x,y)}{p_X(x)} = \frac{p_{X,Y}(x,y)}{\int_y p_{X,Y}(x,y) dy}$$

However: this might be hard/intractable ...

Idea behind supervised learning

- Instead of finding a model for $p_{X,Y}(x,y)$, learn a model for $p_{Y|X}(y|x)$ directly!
- Learn a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ that has a **low generalization error** for new and unseen $x \in \mathcal{X}$

Generalization error / risk minimization

- Given a loss function $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_0^+$
- The **generalization error** (theoretical risk) is expected loss

$$R[f] = \int_{\mathcal{X} \times \mathcal{Y}} \ell(f(x), y) p_{\mathcal{X}, \mathcal{Y}}(x, y) dx dy$$

- We want to minimize this wrt. f
- But: joint probability $p_{\mathcal{X}, \mathcal{Y}}(x, y)$ is unknown and $\inf_f R[f]$ cannot be computed directly
- Solution: approximate $R[f]$ using an N -sample (training set)

$$\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N,$$

drawn **independent and identically distributed (iid)** from $p_{\mathcal{X}, \mathcal{Y}}(x, y)$

Empirical Risk Minimization

Empirical Risk Minimization

Idea: minimize the **empirical risk** on $\mathcal{D}$

$$\hat{R}[f] = \frac{1}{N} \sum_{n=1}^N \ell(f(x_n), y_n)$$

Converges asymptotically to $R[f]$ in the limit $N \rightarrow \infty$

Drawbacks:

- Convergence $\hat{R}[f] \rightarrow R[f]$ slow, requires large N
- In general no unique minimum (possibly poor generalization, no distinguished solution)

Linear Regression

Linear Regression

Regression problems involve real-valued response variables $y \in \mathbb{R}$.

A standard linear regression model with Gaussian noise $\epsilon \sim \mathcal{N}(0, s^2)$ is given by

$$f(x) = w^\top x + b \quad \text{and} \quad y = f(x) + \epsilon.$$

Goal: learn $f(x)$ from training data $\mathcal{D}$.

For simplicity, we augment bias b into w as follows. Because of the linearity of $f(x) = w^\top x + b$, it holds

$$\begin{aligned} f(x) &= w^\top x + b \\ &= \left(\sum_{d=1}^D w_d x_d \right) + b \\ &= \left(\sum_{d=1}^D w_d x_d \right) + \underbrace{b}_{=:w_0} \times \underbrace{1}_{=:x_0} \\ &= \sum_{d=0}^D w_d x_d = \tilde{w}^\top \tilde{x} \end{aligned}$$

where $\tilde{w} = (b, w_1, w_2, \dots, w_D)^\top$ and $\tilde{x} = (1, x_1, \dots, x_D)^\top$.

To not clutter notation unnecessarily, we drop the extra $\sim$ and simply write $f(x) = w^\top x$.

Sketch

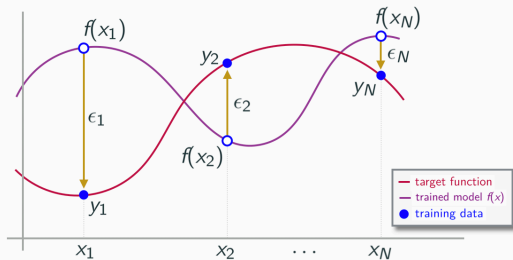


Figure 2: Visualization of **linear** regression

Note:

- That is **linear** in the parameters
- We can introduce arbitrary nonlinear transformation $\phi(x)$
- E.g. $\phi(x) = [1, x^1, x^2, \dots, x^{K-1}]^T$ for $x \in \mathbb{R}$
- And use $f(x) = w^T \phi(x)$ instead of $f(x) = w^T x$

Likelihood

Recall that ϵ is zero-mean Gaussian distributed. Since

$$y = f(x) + \epsilon \quad \text{implies} \quad \epsilon = y - f(x)$$

it follows that $y - f(x)$ is also Gaussian ($\mathcal{N}(0, s^2)$) distributed

The likelihood of $y = (y_1, \dots, y_N)^\top$ is thus given by

$$\begin{aligned} L(y|x_1, \dots, x_N) &= \prod_{n=1}^N p(y_n|x_n; w) \\ &= \prod_{n=1}^N \mathcal{N}(y_n|w^\top x_n, s^2) \\ &= \prod_{n=1}^N \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{(y_n - w^\top x_n)^2}{2s^2}\right) \end{aligned}$$

With compact notation

$$\begin{aligned} L(y|x_1, \dots, x_N) &= \prod_{n=1}^N p(y_n|x_n; w) \\ &= \prod_{n=1}^N \mathcal{N}(y_n|w^\top x_n, s^2) \\ &= \prod_{n=1}^N \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{(y_n - w^\top x_n)^2}{2s^2}\right) \\ &= \frac{1}{(2\pi s^2)^{N/2}} \exp\left(-\frac{1}{2s^2} \|y - Xw\|^2\right) \end{aligned}$$

- X is called the **design matrix**, obtained by stacking up training examples row-wise, such that n th row contains n th example.

Stack inputs into **design matrix** $X = \begin{bmatrix} -x_1- \\ \vdots \\ -x_N- \end{bmatrix} \in \mathbb{R}^{N \times D}$

Remember $y = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N$

(beware: *overloaded* notation)

Log-likelihood

Instead of maximizing the likelihood L directly, we maximize the log-likelihood $\log L$ instead:

$$\begin{aligned}\log L(y|x_1, \dots, x_N) &= \log \left(\frac{1}{(2\pi s^2)^{N/2}} \exp \left(-\frac{1}{2s^2} \|y - Xw\|^2 \right) \right) \\ &= \underbrace{\log \frac{1}{(2\pi s^2)^{N/2}}}_{\text{constant}} + \left(-\frac{1}{2s^2} \|y - Xw\|^2 \right) \\ &= -\frac{1}{2s^2} \|y - Xw\|^2 + \text{constant}\end{aligned}$$

Maximizing the likelihood thus leads to the optimization problem

$$\operatorname{argmax}_w -\frac{1}{2s^2} \|y - Xw\|^2.$$

We drop the s^2 as it does not change the $\operatorname{argmax} \dots$

Minimization problem

...and turn the problem into the equivalent minimization problem

$$\operatorname{argmin}_w \frac{1}{2} \|y - Xw\|^2$$

Note that $\|y - Xw\|^2 = \|\epsilon\|^2$: Linear regression minimizes the squared Euclidean distance between predictions $f(x)$ and target variables y .

Using the **squared loss**, given by $\ell_{sqrd}(x, y, f) = (y - f(x))^2$, allows to cast linear regression into the empirical risk minimization framework

$$\operatorname{argmin}_w \frac{1}{2} \sum_{n=1}^N \ell_{sqrd}(x_n, y_n, f) = \operatorname{argmin}_w \frac{1}{2} \sum_{n=1}^N (y_n - f(x_n))^2$$

Remember sketch

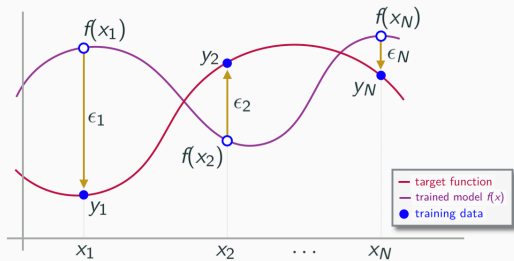


Figure 3: Visualization of linear regression

Squared loss function

- $\leadsto$ Visualization
- In the context of linear regression: convex in the parameters
- Convex optimization

Closed form solution

To minimize the objective, we compute the gradient using the chain rule

$$\begin{aligned}\frac{\partial}{\partial w} \frac{1}{2} \|Xw - y\|^2 &= \frac{2}{2} X^\top (Xw - y) \\ &= X^\top Xw - X^\top y\end{aligned}$$

Setting the derivative to zero and solving for w gives

$$\begin{aligned}X^\top Xw - X^\top y &= 0 \\ X^\top Xw &= X^\top y \\ w &= (X^\top X)^{-1} X^\top y\end{aligned}$$

where $(X^\top X)^{-1}$ denotes the inverse of $(X^\top X)$

The optimal solution can be computed in closed-form!

But: the inverse $(X^T X)^{-1}$ only exists if $(X^T X)$ has full rank!

Alternatively, the inverse needs to be approximated by a pseudo inverse (most of the optimizers will switch automatically to computing pseudo inverse in that case).

However, keep in mind that mathematically, a solution does not exist if $\text{rank}(X^T X) < D$.

Another alternative: gradient descent

In general: min. objective function $E(w) = \sum_{n=1}^N E_n(w)$

Can be solved with gradient descent, if gradient wrt. w , $\nabla E(w)$ can be computed:

Initialize $w^{(1)}$ (randomly)

for $t = 1, 2, \dots$

$$w^{(t+1)} = w^{(t)} - \alpha^{(t)} \nabla E(w^{(t)})$$

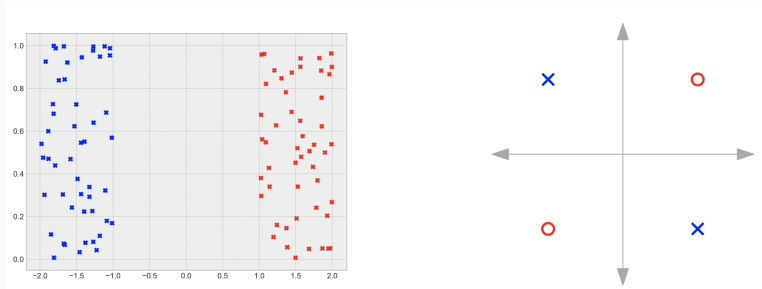
where $\alpha^{(t)} > 0$ is the learning rate (that can depend on the step).

Logistic Regression

Linear regression: Given labeled data $\{(x_n, y_n)\}$ with $x_n \in \mathbb{R}$ and $y_n \in \mathbb{R}$, minimize **empirical risk** with squared loss $V_{squared}$ and hypothesis $f(x) = w^\top x + b$ by **fitting the training data** (regression)

Logistic regression: Given labeled data $\{(x_n, y_n)\}$ with $x_n \in \mathbb{R}$ and $y_n \in \{0, 1\}$, minimize **empirical risk** with logistic loss $V_{logistic}$ and hypothesis $f(x) = w^\top x + b$ by **fitting the training data** (classification by regression)

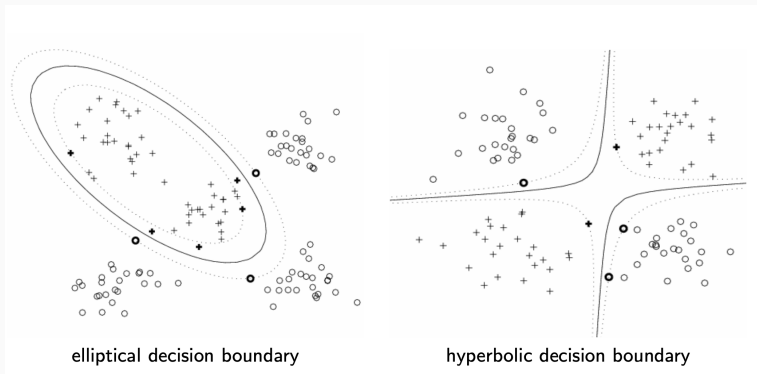
Linear separability



Is it always possible to find parameters (w, b) such that a classifier $f(x) = w^\top x + b$ predicts always the correct class, e.g. $\sigma(f(x)) = y$?

Feature expansion

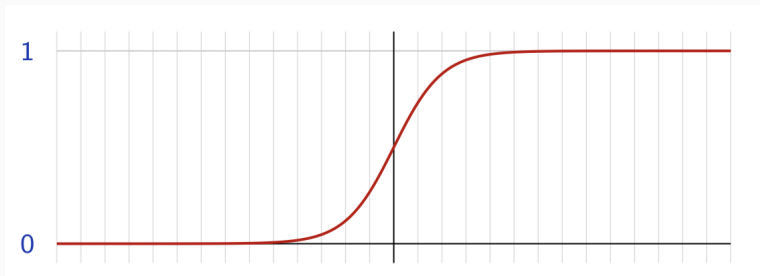
We can map inputs $x_1, \dots, x_N$ to exotic inner product spaces and realize complex decision boundaries.



The logistic function

We aim to output probabilities $P(y = 1|x)$ but don't want to leave our linear model. We thus use a sigmoid function σ to squash the outputs of $z = f(x)$ into the interval $[0, 1]$.

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + \exp(-z)}$$



We have

$$\begin{aligned}P(y = 1|x) &= \sigma(f(x)) \\&= \sigma(w^\top x + b) \\&= \frac{1}{1 + \exp(-(w^\top x + b))}\end{aligned}$$

and

$$\begin{aligned}P(y = 0|x) &= 1 - P(y = 1|x) \\&= 1 - \sigma(w^\top x + b) \\&= 1 - \frac{1}{1 + \exp(-(w^\top x + b))} \\&= \frac{\exp(-(w^\top x + b))}{1 + \exp(-(w^\top x + b))}\end{aligned}$$

Note that the sigmoid realizes $1 - \sigma(z) = \sigma(-z)$.

Why not regression?

Recall squared loss for instance (x_n, y_n) :

$$V_{squared} = (y_n - \underbrace{f(x_n)}_{=:\hat{y}})^2$$

Squared loss aims at equivalence of prediction $\hat{y}$ and true label y_n .

In classification we will decide for the most probable label. In case of a wrong prediction, we just need to push the example across the “class border” rather than optimizing until equivalence of $\hat{y}$ and y_n .

Using (linear) regression will likely generate function values out of bounds (e.g., $f(x_n) \notin [0, 1]$). Does not make sense if the output is expected to allow for an interpretation as probability.

Bernoulli distribution

When there are only two possible outcomes of an experiment (e.g., a coin flip), the Bernoulli distribution can often be applied:

Let $Y = 1$ for outcome *head* and $Y = 0$ for outcome *tail*, assume that the coin may not be a fair one. Using the Bernoulli distribution, we express the probability of observing $y \in \{0, 1\}$ by

$$P(Y = y) = \begin{cases} \beta & : \text{if } y = 1 \\ 1 - \beta & : \text{if } y = 0 \end{cases}$$

with $\beta \in [0, 1]$. The above equation can be simplified into a single line by

$$p(y) = \beta^y (1 - \beta)^{1-y}.$$

Note that in case of logistic regression, the prediction $f(x) = \hat{y}$ takes the role of the probability β .

Logistic loss

We derive the loss function for a single observation x as follows:
Since there are only two (discrete) labels, we begin with the Bernoulli distribution

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

where $\hat{y} = \sigma(f(x))$ and true label $y \in \{0, 1\}$. We take the log (recall: does not change maximizer!) and obtain the log likelihood

$$\log p(y|x) = \log (\hat{y}^y (1 - \hat{y})^{1-y}) = y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

We aim to maximize this expression. To turn it into a loss function that can be minimized, we thus flip the sign and obtain the **cross-entropy** loss

$$\begin{aligned} V_{ce}(x, y, f) &= -\log p(y|x) \\ &= -(y \log [\sigma(f(x))] + (1 - y) \log [1 - \sigma(f(x))]) \end{aligned}$$

Since $y \in \{0, 1\}$, only one of the terms (red or pink) remains while the other is zero

Hence, the loss measures the probability of an observation belonging to the correct class and ignores the other class(es).

Hence, a simpler version of the loss function can be obtained for considering labels $\tilde{y} \in \{+1, -1\}$ rather than $y \in \{0, 1\}$,

$$P(\tilde{y}|x) = \sigma(\tilde{y} f(x)) = \sigma(\tilde{y} (w^\top x + b)) = \frac{1}{1 + \exp(-\tilde{y} (w^\top x + b))}$$

Using the above result for $\tilde{y} \in \{+1, -1\}$, the cross-entropy simplifies to the **logistic** loss V_{logistic}

$$\begin{aligned} V_{\text{logistic}}(x, \tilde{y}, f) &= -\log p(\tilde{y}|x) \\ &= -\log \frac{1}{1 + \exp(-\tilde{y} (w^\top x + b))} \\ &= -(\log(1) - \log(1 + \exp(-\tilde{y} (w^\top x + b)))) \\ &= \log(1 + \exp(-\tilde{y} f(x))) \end{aligned}$$

Empirical Risk Minimization (ERM)

Using ERM together with cross-entropy loss yields the objective

$$\hat{R}_{logreg}[f] = \frac{1}{N} \sum_{n=1}^N V_{logistic}(x_n, \tilde{y}_n, f)$$

that we aim to minimize w.r.t. w and b .

↪ with gradient descent

The derivative $\nabla_w \hat{R}_{logreg}$ is given by

$$\begin{aligned}\frac{\partial R_{logreg}}{\partial w} &= \frac{1}{N} \sum_{n=1}^N \nabla_w V_{logistic}(x_n, \tilde{y}_n, f) \\ &= \frac{1}{N} \sum_{n=1}^N \frac{1}{1 + \exp(-\tilde{y}_n f(x_n))} \exp(-\tilde{y}_n f(x_n)) (-\tilde{y}_n) \nabla_w f(x_n) \\ &= -\frac{1}{N} \sum_{n=1}^N \frac{\tilde{y}_n x_n}{1 + \exp(\tilde{y}_n f(x_n))}\end{aligned}$$

yielding the (batch) gradient update

$$w^{n+1} \leftarrow w^n + \eta_n \frac{1}{N} \sum_{n=1}^N \frac{\tilde{y}_n x_n}{1 + \exp(\tilde{y}_n f(x_n))}$$

The derivative $\nabla_b \hat{R}_{logreg}$ is analogous/similar.

Perceptrons

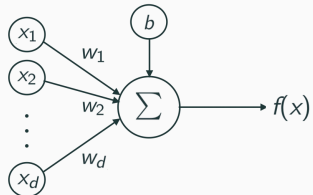
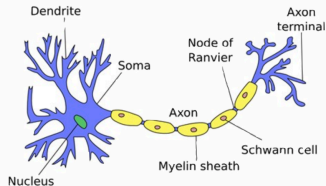
Perceptrons (Rosenblatt, 1957)

The **perceptron** is a binary classifier $f: \mathbb{R}^D \rightarrow \{-1, +1\}$

Its decision function is given by

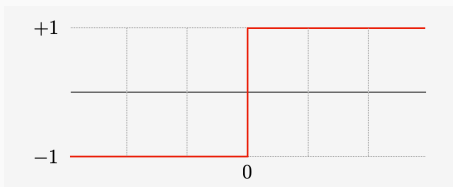
$$f(x) = \sum_{d=1}^D w_d x_d + b = w^\top x + b$$

Originally motivated by biology, with w_d being the synaptic weights, inputs x_d and firing rates f . It is a crude biological model:



Since responses are in $\{-1, +1\}$, we introduce the sign function σ

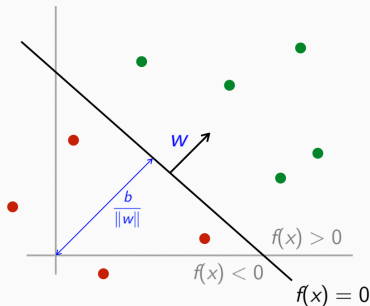
$$\sigma(z) = \begin{cases} +1 & : z \geq 0 \\ -1 & : \textit{otherwise} \end{cases}$$



Using σ , the perceptron classification rule boils down to

$$\sigma(f(x)) = \sigma(w^T x + b) = \hat{y}$$

The perceptron realizes a hyperplane in input space with normal vector w and distance $b/\|w\|$ from the origin



Instances of the positive class ($y = +1$) should lie in the positive halfspace ($f(x) > 0$), instances of the negative class ($y = -1$) in the negative halfspace ($f(x) < 0$), respectively

Relation to empirical risk minimization

The loss function of the perceptron is given by

$$\hat{R} = \sum_{n=1}^N V_{\text{perceptron}}(x_n, y_n, f) = - \sum_{n=1}^N y_n f(x_n)$$

Instead of processing all instances in a batch, the perceptron is updated after seeing every instance (see next slide).

Computing the derivatives wrt w and b yields

$$\frac{\partial \hat{R}}{\partial w} = \sum_{i=1}^N -y_n x_n \quad \text{and} \quad \frac{\partial \hat{R}}{\partial b} = \sum_{i=1}^N -y_n$$

However, the perceptron is updated after every mistake. An example of *stochastic gradient descent*. The gradients are given by

$$\frac{\partial V_{\text{perc}}}{\partial w} = -y_n x_n \quad \text{and} \quad \frac{\partial V_{\text{perc}}}{\partial b} = -y_n$$

Batch vs Online Learning

Batch learning uses the full gradient (batch), that is, we run over the entire training set and collect all gradients and perform a single update step using the **true gradient** afterwards:

$$\frac{\partial}{\partial w} \sum_{n=1}^N V(x_n, y_n, f)$$

Online learning updates the model after every instance (or mistake) directly. We use the loss function to compute the gradient

$$\frac{\partial}{\partial w} V(x_n, y_n, f).$$

Due the (random) ordering of the training instances, this scheme resembles a stochastic process and is called **stochastic gradient descent (SGD)**.

Perceptron Learning Rule

Given a training set

$$(x_n, y_n) \in \mathbb{R}^D \times \{-1, +1\}, n = 1, \dots, N,$$

the perceptron algorithm adapts the parameters w, b to the data at-hand:

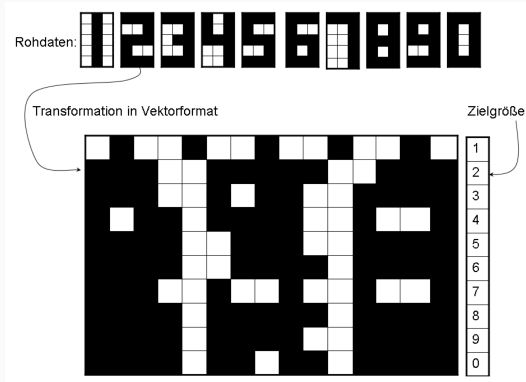
1. Start with $w^0 = \mathbf{0}$
2. while $\exists n_k$ s.t. $y_{n_k} ((w^k)^\top x_{n_k}) \leq 0$, update $w^{k+1} = w^k + y_{n_k} x_{n_k}$

The bias b can be augmented into w by adding a constant component to x equal to 1, such that $\tilde{w} = (b, w_1, \dots, w_D)^\top$ and $\tilde{x} = (1, x_1, \dots, x_D)^\top$

Alternatively, the update rule for b is given by $b^{k+1} = b^k + y_{n_k}$

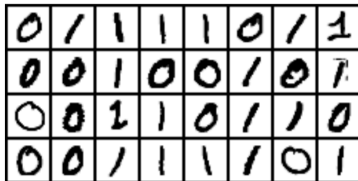
Example: OCR

Transforming data into design matrix:



(Thanks to Volker Tresp for the slide)

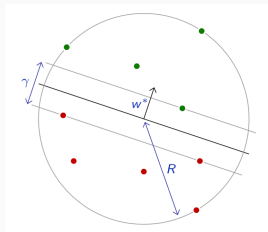
The perceptron algorithm works often surprisingly well. With MNIST's '0's as negative class, and '1's as positive one.



```
epoch 0 nb_changes 64 train_error 0.23% test_error 0.19%
epoch 1 nb_changes 24 train_error 0.07% test_error 0.00%
epoch 2 nb_changes 10 train_error 0.06% test_error 0.05%
epoch 3 nb_changes 6 train_error 0.03% test_error 0.14%
epoch 4 nb_changes 5 train_error 0.03% test_error 0.09%
epoch 5 nb_changes 4 train_error 0.02% test_error 0.14%
epoch 6 nb_changes 3 train_error 0.01% test_error 0.14%
epoch 7 nb_changes 2 train_error 0.00% test_error 0.14%
epoch 8 nb_changes 0 train_error 0.00% test_error 0.14%
```

Convergence

We can get a convergence result under two assumptions:



1. The x_n are in a sphere of radius R :

$$\exists R > 0 : \forall n \quad \|x_n\| \leq R$$

2. The two populations can be separated with a margin $\gamma > 0$:

$$\exists w^* > 0, \|w^*\| = 1, \exists \gamma > 0 : \forall n \quad y_n((w^*)^\top x_n) \geq \gamma/2$$

To prove the convergence, let us make the assumption that there still is a misclassified sample at iteration k , and w^{k+1} is the weight vector updated with it. We have

$$\begin{aligned}(w^{k+1})^\top w^* &= (w^k + y_{n_k} x_{n_k})^\top w^* \\&= (w^k)^\top w^* + y_{n_k} ((x_{n_k})^\top w^*) \\&\geq (w^k)^\top w^* + \gamma/2 \\&\geq (k+1)\gamma/2\end{aligned}$$

Since

$$\|w^k\| \|w^*\| \geq (w^k)^\top w^*$$

we get

$$\begin{aligned}\|w^k\|^2 &\geq ((w^k)^\top w^*)^2 / \|w^*\|^2 \\&\geq k^2 \gamma^2 / 4\end{aligned}$$

And

$$\begin{aligned}\|w^{k+1}\|^2 &= (w^{k+1})^\top w^{k+1} \\&= (w^k + y_{n_k} x_{n_k})^\top (w^k + y_{n_k} x_{n_k}) \\&= (w^k)^\top w^k + \underbrace{2y_{n_k} (w^k)^\top x_{n_k}}_{\leq 0} + \underbrace{\|x_{n_k}\|^2}_{\leq R^2} \\&\leq \|w^k\|^2 + R^2 \\&\leq (k+1)R^2\end{aligned}$$

Putting these two results together, we get

$$k^2\gamma^2/4 \leq \|w^k\|^2 \leq kR^2$$

hence

$$k \leq 4R^2/\gamma^2$$

That is, no misclassified sample can remain after $\lfloor 4R^2/\gamma^2 \rfloor$ iterations.

This result makes sense:

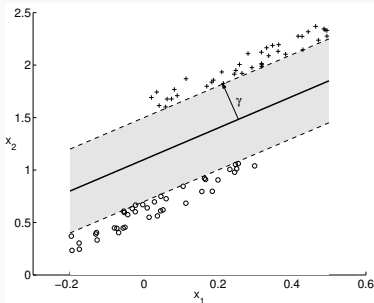
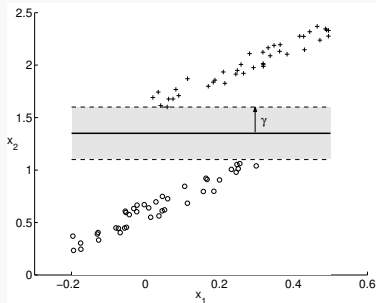
1. The bound does not change if the population is scaled, and
2. the larger the margin, the more quickly the algorithm classifies all the samples correctly.

Introduction to Machine Learning

Yannick Rudolph, Ulf Brefeld

build: January 31, 2025

Machine Learning Group
Leuphana University of Lüneburg



Left: a suboptimal separation

Right: the optimal decision hyperplane characterized by a maximal **geometrical margin** γ (see next slides)

- solid line: decision hyperplane $w^T x + b = 0$
- dashed lines: corridor hyperplanes $w^T x + b = \pm \gamma$

Margins

Margins are measures for distances of training instances (x_n, y_n) and a decision hyperplane $f(x)$

The **functional margin** $\tilde{\gamma}$ is the decision value $\tilde{\gamma} = y_n(w^\top x_n + b)$

The functional margin depends on the scaling of w : Multiplying w with a positive constant does not change the classification

The **geometrical margin** $\gamma = \tilde{\gamma} / \|w\|$ remedies the dependence

γ equals the Euclidean distance of to the decision hyperplane

The hyperplane that maximizes γ for all training points is called the **optimal decision hyperplane**

Given training data $\mathcal{D}$ and a linear decision function $f(x) = w^\top x + b$:

- The functional margin $\tilde{\gamma}_n$ of instance n is given by

$$\tilde{\gamma}_n = y_n (w^\top x_n + b)$$

- The functional margin $\tilde{\gamma}_{\mathcal{D}}$ of the set $\mathcal{D}$ is given by

$$\tilde{\gamma}_{\mathcal{D}} = \min_n \tilde{\gamma}_n$$

- The geometrical margin γ_n of instance n is given by

$$\gamma_n = \tilde{\gamma}_n / \|w\|$$

- The geometrical margin $\gamma_{\mathcal{D}}$ of the set $\mathcal{D}$ is given by

$$\gamma_{\mathcal{D}} = \tilde{\gamma}_{\mathcal{D}} / \|w\|$$

From the definition, it follows that correctly classified instances (x_n, y_n) imply $\tilde{\gamma}_n > 0$

Since $\|w\| > 0$, this holds analogously also for $\gamma_n > 0$

For $\|w\| = 1$, both functional and geometrical margins are identical
(Compare the concept of margin also with the perceptron convergence proof)

To construct the optimal decision hyperplane, assume that $\mathcal{D}$ is linearly separable and that we already have an initial solution $f(x) = w^\top x + b$ that does not realize the maximal margin

Split training data into a positive $\mathcal{D}_{+1}$ and negative $\mathcal{D}_{-1}$ set

$$\mathcal{D}_y = \{(x_n, y_n) : (x_n, y_n) \in \mathcal{D}, y_n = y\}, \quad y = \pm 1$$

such that $\mathcal{D} = \mathcal{D}_{+1} \cup \mathcal{D}_{-1}$ and $\mathcal{D}_{+1} \cap \mathcal{D}_{-1} = \emptyset$

Idea: Place corridor hyperplanes around f such that no instance lies in the corridor

- the bigger the corridor, the better the separation
- maximize the width of the corridor!

Compute functional margins $\tilde{\gamma}_{\pm 1}$ for the sets $\mathcal{D}_{\pm 1}$ given f ,

$$\tilde{\gamma}_{+1} = \min_{(x_n, y_n) \in \mathcal{D}_{+1}} y_n (w^\top x_n + b)$$

$$\tilde{\gamma}_{-1} = \min_{(x_n, y_n) \in \mathcal{D}_{-1}} y_n (w^\top x_n + b)$$

The corridor hyperplanes are given by

$$w^\top x + b = +\tilde{\gamma}_{+1}$$

$$w^\top x + b = -\tilde{\gamma}_{-1}$$

The width of the corridor is $\gamma_{+1} + \gamma_{-1}$

The hyperplane in the center of the corridor is given by

$$w^\top x + b = \frac{\tilde{\gamma}_{+1} - \tilde{\gamma}_{-1}}{2} =: b_0$$

and has a functional margin of $\tilde{\gamma} = \frac{\tilde{\gamma}_{+1} + \tilde{\gamma}_{-1}}{2}$

The corridor hyperplanes can now be expressed relative to the center hyperplane as

$$w^\top x + b = b_0 + \tilde{\gamma}$$

$$w^\top x + b = b_0 - \tilde{\gamma}$$

Multiplying the equations with a positive constant does not change the hyperplanes or classification

We aim for a unique solution and divide by $\tilde{\gamma}$ (recall that $\tilde{\gamma} > 0$)

Using $\tilde{w} = \frac{1}{\tilde{\gamma}} w$ and $\tilde{b} = \frac{b-b_0}{\tilde{\gamma}}$ we obtain simplified corridor hyperplanes

$$\tilde{w}^\top x + \tilde{b} = +1 \quad \text{and} \quad \tilde{w}^\top x + \tilde{b} = -1,$$

realizing a functional margin $\tilde{\gamma} = 1$

The decision hyperplane can now be written as

$$\tilde{w}^\top x + \tilde{b} = 0$$

Dropping the tildes for ease of notation results in the decision hyperplane

$$f(x) = w^T x + b = 0$$

and the corridor hyperplanes

$$f(x) = w^T x + b = +1$$

$$f(x) = w^T x + b = -1$$

Optimization Problem

Setting $\tilde{\gamma} = 1$ implies a geometrical margin

$$\gamma = \frac{1}{\|w\|}$$

That is, maximizing the geometrical margin γ is equivalent to minimizing $\|w\|$

The optimization problem is

$$\begin{aligned} \min_{w,b} \quad & \|w\|^2 \\ \text{s.t.} \quad & y_n (w^\top x_n + b) \geq 1 \quad \forall 1 \leq n \leq N \end{aligned}$$

The minimum w^* realizes the optimal decision hyperplane with a geometrical margin $\gamma = \frac{1}{\|w\|}$

Lagrange's Theorem

Lagrange's theorem solves optimization problems with constraints

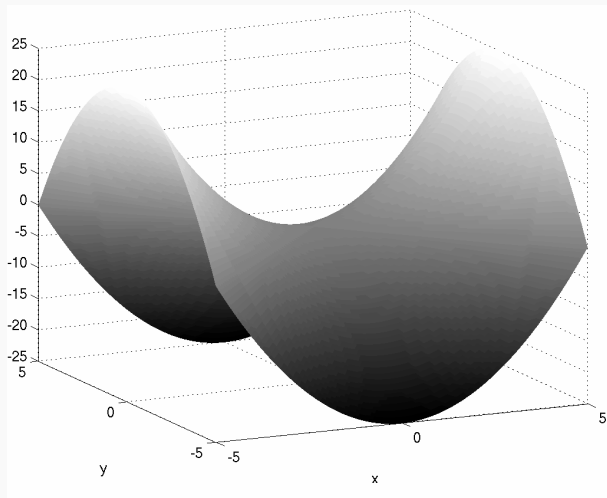
In a nutshell: Every constraint is weighted with a non-negative Lagrange multiplier and added/subtracted to/from the objective function (depends on type of inequality)

The **primal Lagrangian** with multipliers $\alpha_n \geq 0$ is given by

$$L_P(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{n=1}^N \alpha_n [y_n (w^\top x_n + b) - 1]$$

- The Lagrangian needs to be minimized wrt **primal** variables w, b and maximized wrt **dual** variables $\alpha = (\alpha_1, \dots, \alpha_N)^\top$
- The objective is convex and quadratic

⇒ The optimum is a saddle point



Saddle point of $f(x, y) = x^2 - y^2$ at position $(0, 0)$. It holds

$$f(0, y) \leq f(0, 0) \leq f(x, 0)$$

Goal of optimization: eliminate primal variables w, b so that only dual variables α remain

Maximize L_P wrt α subject to $\alpha_i \geq 0$ and

$$\frac{\partial L_P}{\partial w} = w - \sum_{n=1}^N \alpha_n y_n x_n \stackrel{!}{=} \mathbf{0} \quad \Rightarrow \quad w^* = \sum_{n=1}^N \alpha_n y_n x_n$$

$$\frac{\partial L_P}{\partial b} = - \sum_{n=1}^N \alpha_n y_n \stackrel{!}{=} 0 \quad \Rightarrow \quad 0 = \sum_{n=1}^N \alpha_n y_n$$

Dual Representation

Plugging the derivatives w^* and b^* into the objective leads to the **dual Lagrangian** L_D (Wolfe dual) that depends only on dual variables α

$$\begin{aligned} L_P(w, b, \alpha) &= \frac{1}{2} \|w\|^2 - \sum_{n=1}^N \alpha_n [y_n (w^\top x_n + b) - 1] \\ &= \frac{1}{2} \|w\|^2 - \sum_{n=1}^N \alpha_n y_n (w^\top x_n + b) + \sum_{n=1}^N \alpha_n \\ &= \frac{1}{2} \left(\sum_{n=1}^N \alpha_n y_n x_n \right)^2 - \left(\sum_n \alpha_n y_n x_n \right)^2 + \sum_{n=1}^N \alpha_n \\ L_D(\alpha) &= \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n,m=1}^N \alpha_n \alpha_m y_n y_m x_n^\top x_m \end{aligned}$$

Note: L_D depends only on linear combinations of training instances

$$L_D(\alpha) = \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n,m=1}^N \alpha_n \alpha_m y_n y_m x_n^\top x_m$$

is the simplest version of a **support vector machine** (SVM)
(works only for linearly separable data by maximizing the margin)

The solution α^* fulfills the Karush Kuhn Tucker (KKT) conditions:

$$\begin{aligned} \frac{\partial L_P(w^*, b^*, \alpha^*)}{\partial w} &= 0 \\ \frac{\partial L_P(w^*, b^*, \alpha^*)}{\partial b} &= 0 \\ \alpha_n^* [y_n (w^{*\top} x_n + b^*) - 1] &= 0 \\ y_n (w^{*\top} x_n + b^*) - 1 &\geq 0 \\ \alpha_n^* &\geq 0 \end{aligned}$$

Support Vectors

The KKT conditions state that in the optimum

$$\alpha_n^* [y_n (w^{*\top} x_n + b^*) - 1] = 0$$

That is, for every n , either

$$\alpha_n = 0 \quad \text{or} \quad y_n \underbrace{(w^{*\top} x_n + b^*)}_{f^*(x_n)} - 1 = 0$$

Recall: $y_n f^*(x_n) = 1$ is only fulfilled for points x_n lying exactly on one of the corridor hyperplanes

That is, these points realize the smallest margin and the solution w^*, b^* would change if we changed these points

These points are thus called **support vectors**

Defining the set of support vectors

$$\mathcal{S} = \{(x_n, y_n) : (x_n, y_n) \in \mathcal{D}, \alpha_n^* > 0\}$$

Since for all $x_n \notin \mathcal{S}$ the corresponding $\alpha_n^* = 0$ we have

$$w^* = \sum_{n=1}^N \alpha_n^* y_n x_n = \sum_{(x_n, y_n) \in \mathcal{S}} \alpha_n^* y_n x_n$$

The optimal solution thus depends only on a few training points, the support vectors, and we say the solution is **sparse** as most of the α^* are zero

Without this property, data at large scales would not be feasible for an SVM

Threshold b

In contrast to w^* , the threshold b^* is not computed explicitly in the optimization

Again, the KKT conditions help out and suggest to compute b^* for an $\alpha_n > 0$. A more robust way to compute b^* is via averaging

$$\begin{aligned} b^* &= \frac{1}{|\mathcal{S}|} \sum_{(x_n, y_n) \in \mathcal{S}} (y_n - w^{*\top} x_n) \\ &= \frac{1}{|\mathcal{S}|} \sum_{(x_n, y_n) \in \mathcal{S}} \left[y_n - \sum_{m=1}^N \alpha_m^* y_m x_m^\top x_n \right] \end{aligned}$$

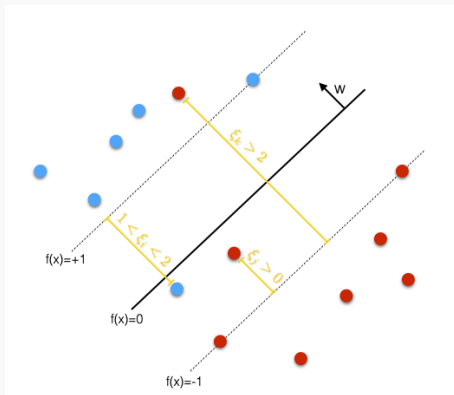
Once the solution w^* or α^* and b^* are found, new and so far unseen instances x can be classified into $\{+1, -1\}$ by

$$\begin{aligned} f(x) &= \text{sign} \left[w^{*\top} x + b^* \right] \\ &= \text{sign} \left[\sum_{(x_n, y_n) \in \mathcal{S}} \alpha_n^* y_n x_n^\top x + b^* \right] \end{aligned}$$

Support Vector Machines

Back to reality...

Real data is usually not linearly separable but noisy and non-linear



Generalization: take (small) errors ('slacks' ξ) into account

So-called slack variables $\xi_n \geq 0$ compensate for local errors and allow for point-wise relaxations of the margin

They are included in the constraints as follows

$$\begin{array}{ll} w^\top x_n + b \geq +1 - \xi_n & \text{for } (x_n, y_n) \in \mathcal{D}_{+1} \\ w^\top x_n + b \leq -1 + \xi_n & \text{for } (x_n, y_n) \in \mathcal{D}_{-1} \\ \xi_n \geq 0 & \text{for } 1 \leq n \leq N \end{array}$$

The first two constraints can be unified by multiplying with y_n as before. We obtain for all $1 \leq n \leq N$

$$\begin{array}{l} y_n (w^\top x_n + b) \geq +1 - \xi_n \\ \xi_n \geq 0 \end{array}$$

The value of ξ_n determines the geometrical location of the corresponding training instance (x_n, y_n) (see next slide)

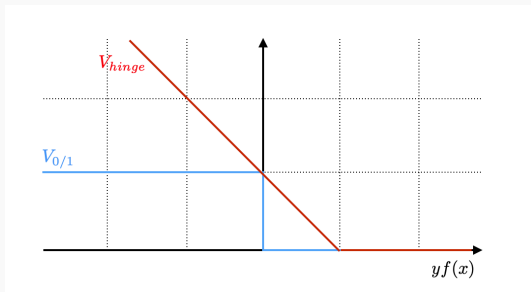
We differentiate four cases:

- if $\xi_n = 0$, the instance is correctly classified
- if $0 < \xi_n < 1$, the instance is correctly classified but lies in the corridor
- if $\xi_i = 1$, the instance lies on the decision hyperplane, and
- if $\xi_i > 1$ the instance is lying on the wrong side of the hyperplane

Since wrongly classified instances imply $\xi_n > 1$, the sum of slacks upper bounds the 0/1-loss,

$$\sum_{n=1}^N \xi_n \geq \sum_{n=1}^N V_{0/1}(y_n, x_n, f)$$

Visual comparison of hinge loss and 0/1 loss:



$$\begin{aligned} y_n (w^\top x_n + b) \geq 1 - \xi_n &\Rightarrow \xi_n = \max \{0, 1 - y_n f(x_n)\} \\ &\Rightarrow V_{hinge}(x, y, f) = \max \{0, 1 - yf(x)\} \end{aligned}$$

SVMs: primal optimization problem

Putting things together leads to a convex and quadratic problem

$$\min_{w,b,\xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{n=1}^N \xi_n \quad (1)$$

$$\text{s.t.} \quad y_n (w^\top x_n + b) \geq 1 - \xi_n \quad (2)$$

$$\xi_n \geq 0 \quad (3)$$

Trade-off $C > 0$ balances margin maximization and error minimization:

- Large values of C focus on minimizing the error
- Small values of C focus on maximizing the margin
- Find best value in **model selection**

The primal Lagrangian $L_P(w, b, \xi, \alpha, \mu)$ is given by

$$\frac{1}{2} \|w\|^2 + C \sum_{n=1}^N \xi_n - \sum_{n=1}^N \alpha_n [y_n (w^\top x_n + b) - 1 + \xi_n] - \sum_{n=1}^N \mu_n \xi_n,$$

with Lagrange multipliers $\alpha = (\dots, \alpha_n, \dots)^\top$ and $\mu = (\dots, \mu_n, \dots)^\top$.

In the saddle point, the KKT conditions hold (see next slide)

KKT conditions

$$\frac{\partial L_P}{\partial w} = w - \sum_{n=1}^N \alpha_n y_n x_n = \mathbf{0}$$

$$\frac{\partial L_P}{\partial b} = - \sum_{n=1}^N \alpha_n y_n = 0$$

$$\frac{\partial L_P}{\partial \xi_n} = C - \alpha_n - \mu_n = 0$$

$$y_n (w^\top x_n + b) - 1 + \xi_n \geq 0$$

$$\xi_n \geq 0$$

$$\alpha_n \geq 0$$

$$\mu_n \geq 0$$

$$\alpha_n [y_n (w^\top x_n + b) - 1 + \xi_n] = 0$$

$$\xi_n (C - \alpha_n) = 0$$

The dual Lagrangian results from plugging the obtained relations back into the primal,

$$L_D(\alpha) = \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n,m=1}^N \alpha_n \alpha_m y_n y_m x_n^\top x_m$$

KKT Equation $C - \alpha_n - \mu_n = 0$ together with the non-negativity $\mu_n \geq 0$ requires $\alpha_n \leq C$, while $\xi_n > 0$ holds only if $\mu_n = 0$ and implies $\alpha_n = C$

The resulting implicit constraint $0 \leq \alpha_n \leq C$ upper bounds α_n from above by C

The vector α thus lies in an N -dimensional box of length C (that's why the constraint is called 'box constraint')

Quadratic programming

The dual optimization problem can be solved by quadratic programming

$$\begin{aligned} L_D(\alpha) &= \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n,m=1}^N \alpha_n \alpha_m y_n y_m x_n^\top x_m \\ &= \alpha^\top \underbrace{\mathbf{1}}_{\text{linear term}} - \frac{1}{2} \alpha \underbrace{\text{diag}(y) X X^\top \text{diag}(y)}_{\text{quadratic term}} \alpha \end{aligned}$$

subject to $\mathbf{0} \leq \alpha \leq \mathbf{C} \mathbf{1}$

Call a quadratic solver (e.g. quadprog, cplex, ...) with the arguments highlighted in red.

Collecting the support vectors in the set

$$\mathcal{S} = \{(x_n, y_n) : (x_n, y_n) \in \mathcal{D}, \alpha_n^* > 0\}$$

the primal parameters are computed by

$$w^* = \sum_{n=1}^N \alpha_n^* y_n x_n = \sum_{(x_n, y_n) \in \mathcal{S}} \alpha_n^* y_n x_n$$

Again, threshold b^* is computed by averaging:

$$b^* = \frac{1}{|\mathcal{S}|} \sum_{(x_n, y_n) \in \mathcal{S}} \left[y_n - \sum_{m=1}^N \alpha_m y_m x_m^\top x_n \right]$$

Primal vs dual

The dual representation $w = \sum_{n=1}^N \alpha_n x_n$ with some $\alpha \in \mathbb{R}^N$ allows to write the decision function as

$$f(x) = \underbrace{w^\top x + b}_{\text{primal}} = \underbrace{\sum_{n=1}^N \alpha_n x_n^\top x + b}_{\text{dual}}$$

The dual decision function depends only on inner products of (training) points in input space

Differences between primal and dual representation?

Primal vs dual

The dual representation $w = \sum_{n=1}^N \alpha_n x_n$ with some $\alpha \in \mathbb{R}^N$ allows to write the decision function as

$$f(x) = \underbrace{w^\top x + b}_{\text{primal}} = \underbrace{\sum_{n=1}^N \alpha_n x_n^\top x + b}_{\text{dual}}$$

The dual decision function depends only on inner products of (training) points in input space

Number of parameters: primal has $w \in \mathbb{R}^D$, the dual $\alpha \in \mathbb{R}^N$. Hence, if $D \ll N$ use the primal (large-scale), otherwise the dual representation (small-scale but many attributes/features)

Dual depends only on inner products of input examples

Recall inner product

$$\mathbf{x}^\top \mathbf{x}' = \sum_{d=1}^D x_d x'_d$$

Inner product is equivalent to cosine if $\|\mathbf{x}\| = \sqrt{\mathbf{x}^\top \mathbf{x}} = 1$

Hence, inner products allow to perform all geometrical computations involving angles, lengths, and distances!

Idea: embed data into an inner product space $\mathcal{H}$ by a mapping

$$\phi: \mathcal{X} \rightarrow \mathcal{H}$$

Mapping ϕ can be highly nonlinear, the only requirement on the induced space $\mathcal{H}$ is that there exists an inner product in $\mathcal{H}$

Kernels

Assume, a symmetric function $k : (x, x') \rightarrow \mathbb{R}_0^+$ computes the inner product in space $\mathcal{H}$, that is

$$k(x, x') = \phi(x)^\top \phi(x')$$

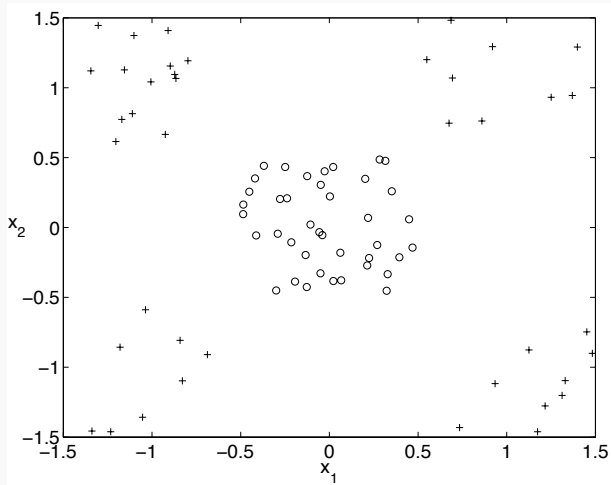
The dual decision function becomes

$$f(x) = \sum_{n=1}^N \alpha_n \phi(x_n)^\top \phi(x) + b = \sum_{n=1}^N \alpha_n k(x_n, x) + b$$

Function k is called a **kernel**:

- transformation $x \mapsto \phi(x)$ is only implicit, we don't need to know ϕ anymore
- k is a similarity measure in $\mathcal{H}$
- useful if data does not live in an inner product space

Example



Data is not linearly separable

Example

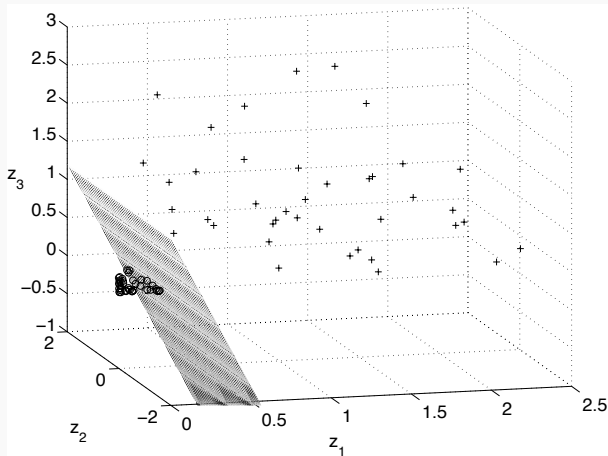
Using $x = (x_1, x_2)^\top$, the explicit transformation $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ is

$$\phi(x) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \end{pmatrix}.$$

For two points $a = (a_1, a_2)^\top$ and $b = (b_1, b_2)^\top$, the inner product $\phi(a)^\top \phi(b)$ in $\mathcal{H}$ can be computed implicitly by a kernel k as follows:

$$\begin{aligned} \phi(a)^\top \phi(b) &= \begin{pmatrix} a_1^2, \sqrt{2} a_1 a_2, a_2^2 \end{pmatrix} \begin{pmatrix} b_1^2 \\ \sqrt{2} b_1 b_2 \\ b_2^2 \end{pmatrix} \\ &= a_1^2 b_1^2 + 2 a_1 a_2 b_1 b_2 + a_2^2 b_2^2 \\ &= \left((a_1, a_2) \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 \\ &= (a^\top b)^2 \\ &= k(a, b). \end{aligned}$$

Example



Using non-linear transformation $\phi: \mathcal{X} \rightarrow \mathcal{H}$, the data becomes linearly separable in $\mathcal{H}$

Which symmetric functions k induce a feature space $\mathcal{H}$ and an implicit mapping $\phi : \mathcal{X} \rightarrow \mathcal{H}$ such that

$$k(x, x') = \phi(x)^\top \phi(x')$$

for all $x, x' \in \mathcal{X}$?

The answer is given by **Mercer's Theorem**. The theorem gives conditions for continuous, symmetric, and positive (semi-)definite functions k on $\mathcal{X} \times \mathcal{X}$ so that they can be written as a convergent series

$$k(x, x') = \sum_{i=1}^{\infty} \lambda_i \phi_i(x) \phi_i(x')$$

for each pair $x, x' \in \mathcal{X} \times \mathcal{X}$

Frequently used kernel functions

- linear kernel: $k(x, x') = x^\top x'$
- d -th order polynomial kernel:

$$k_d(x, x') = (x^\top x')^d \quad \text{or} \quad k_d(x, x') = (x^\top x' + 1)^d$$

- radial basis function (RBF) kernel:

$$k_\sigma(x, x') = \exp \left\{ -\frac{\|x - x'\|^2}{2\sigma^2} \right\}$$

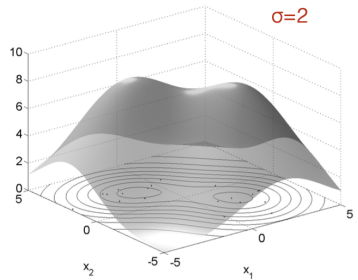
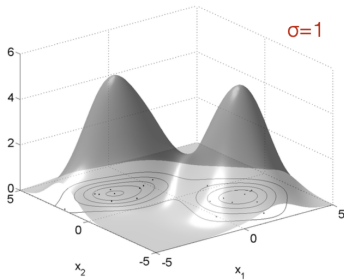
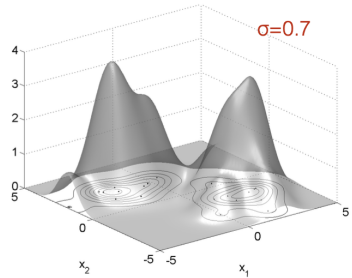
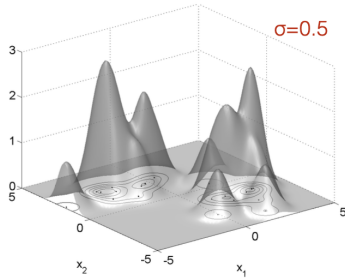
An RBF kernel

$$k_{rbf}(x, x') = \exp \left\{ -\frac{\|x - x'\|^2}{2\sigma^2} \right\}$$

give rise to

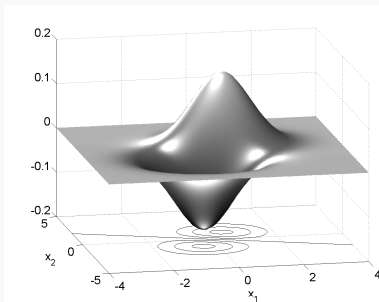
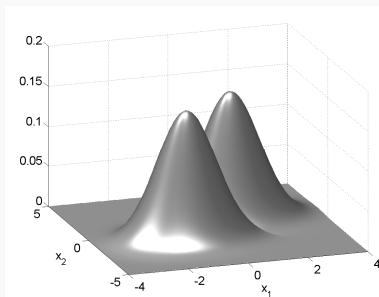
$$f(x) = \sum_{n=1}^N \alpha_n k_{rbf}(x_n, x) = \sum_{n=1}^N \alpha_n \exp \left\{ -\frac{\|x_n - x\|^2}{2\sigma^2} \right\}$$

and the dual decision function is a sum of Gaussians and acts similar to density estimation (see next slide)



SVM decision function

$$f(x) = \sum_{n=1}^N \alpha_n y_n \exp \left\{ -\frac{\|x_n - x\|^2}{2\sigma^2} \right\}$$



Left: density estimation from a different angle. Right: adding labels $y_n \in \{+1, -1\}$ to the equation above

Representer Theorem

Recall regularization

Add a penalty (regularization term Ω) to objective

- induces bias towards particular solutions (e.g., smooth functions vs. peaky functions, sparse functions, etc.)

Minimize the **regularized empirical risk**

$$\hat{R}[f] = \frac{1}{N} \sum_{n=1}^N V(x_n, y_n, f) + \Omega(f)$$

Frequent choice: $\Omega(f) = \|f\|^2 = \|w\|^2$

- Ridge regression
- Support vector machines

Representer Theorem

Representer Theorem: Given a training sample $\{(x_n, y_n)\}_{n=1, \dots, N}$ with $x_n \in \mathbb{R}^D$ and $y_n \in \mathcal{Y}$ and let K be a Mercer kernel on $\mathbb{R}^D$, $V : (\mathbb{R}^D, \mathcal{Y}, \mathcal{H}) \rightarrow \mathbb{R}_0^+$ be an arbitrary but fixed function and $\Omega : \mathbb{R}^D \rightarrow \mathbb{R}$ a non-decreasing function. Let $\mathcal{H}$ be the reproducing kernel Hilbert space induced by K , then any $f \in \mathcal{H}$ minimizing the regularised risk

$$\hat{R}[f] = \frac{1}{N} \sum_{n=1}^N V(x_n, y_n, f) + \Omega(f)$$

admits a representation of the form

$$f^* = \sum_{n=1}^N \alpha_n K(x_n, \cdot)$$

Proof sketch

- Decompose function $f \in \mathcal{F}$ into
 - a part that lies in the span of the inputs

$$\text{span}(\{\phi(x_n) : 1 \leq n \leq N\}) \in \mathcal{F}$$

- and a part $f_{\perp} \in \mathcal{F}$ that is perpendicular to it such that

$$\forall n: \quad \phi(x_n)^{\top} f_{\perp} = 0$$

- Rewrite every $f \in \mathcal{F}$ as $f = \sum_{n=1}^N \alpha_n \phi(x_n) + f_{\perp}$
- Show that the sum $\sum_{n=1}^N V(x_n, y_n, f)$ is minimized by a linear combination of training examples (1st part of proof)
- Show that a linear combination of training examples realizes minimal norms and thus minimizes $\Omega(f)$ (2nd part of proof)

Summary: Regularized Empirical Risk Minimization

Minimizing an objective of the form

$$\hat{R}[f] = \sum_{n=1}^N V(x_n, y_n, f) + \Omega(f),$$

- admits to write the optimal solution as $f^* = \sum_{n=1}^N \alpha_n k(x_n, \cdot)$,
- thus allows for the use of kernels (non-linearity!)
- has a unique solution
- is efficient if $N \ll D$

Many learning algorithms can be phrased as regularized ERM. The methods are also called **kernel machines**

Neural Networks

A tensor is a generalized matrix, a finite table of numerical values indexed along several discrete dimensions.

- A 1d tensor is a vector (e.g. a sound sample),
- A 2d tensor is a matrix (e.g. a grayscale image),
- A 3d tensor is a vector of identically sized matrices (e.g. a multi-channel image),
- A 4d tensor is a matrix of identically sized matrices (e.g. a sequence of multi-channel images),
- etc.

Tensors are used to encode the signal to process, but also the internal states and parameters of the 'neural networks'.

Manipulating data through this constrained structure allows to use CPUs and GPUs at peak performance.

Compounded data structures can represent more diverse data types.

In a nutshell, tensor multiplications allow to express equations like these in a single line of code:

$$\underbrace{\begin{pmatrix} x_1 & y_1 \\ x_2 & y_2 \\ \vdots & \vdots \\ x_N & y_N \end{pmatrix}}_{\text{data} \in \mathbb{R}^{N \times 2}} \quad \underbrace{\begin{pmatrix} x_1 & 1.0 \\ x_2 & 1.0 \\ \vdots & \vdots \\ x_N & 1.0 \end{pmatrix}}_{x \in \mathbb{R}^{N \times 2}} \underbrace{\begin{pmatrix} a \\ b \end{pmatrix}}_{\alpha \in \mathbb{R}^{2 \times 1}} \simeq \underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}}_{y \in \mathbb{R}^{N \times 1}}$$

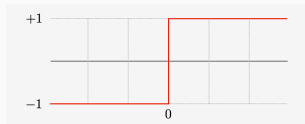
GPUs are optimized for processing tensor multiplications.

Recall: The **perceptron** is a linear model $f: X \rightarrow \{-1, +1\}$

$$f(x) = \sigma(w^\top x + b) = \sigma(w \cdot x + b)$$

with step function σ

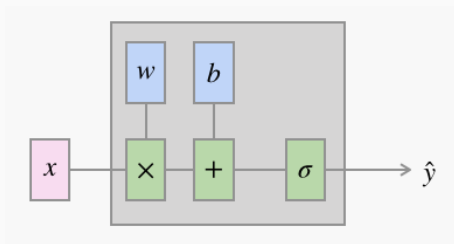
$$\sigma(z) = \begin{cases} +1 & : z \geq 0 \\ -1 & : \textit{otherwise} \end{cases}$$



and realizes a hyperplane in input space with normal vector w and distance $b/\|w\|$ from the origin

In neural networks the function σ is called **activation function**

We will use the following figure to visualize a perceptron

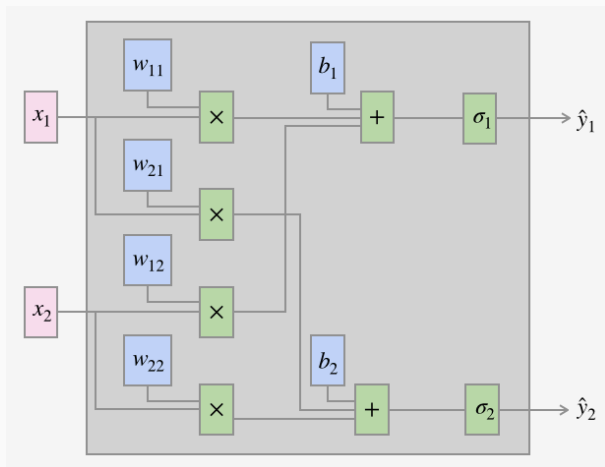


$$f(x) = \sigma(w \cdot x + b) = \hat{y}$$

We stack perceptrons into a 'layer' to produce M outputs where b_i is the **bias** of the i -th unit, and the $w_{i1}, \dots, w_{iN}$ are its **weights**

$$\forall i = 1, \dots, M: \quad \hat{y}_i = \sigma \left(\sum_{j=1}^N w_{ij} x_j + b_i \right)$$

Example for $N = 2, M = 2$:



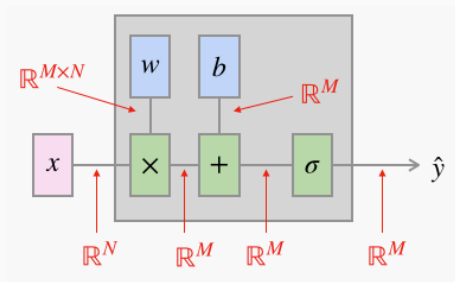
We now have $x \in \mathbb{R}^N$, $w \in \mathbb{R}^{M \times N}$, $b \in \mathbb{R}^M$, $y \in \mathbb{R}^M$, and extend σ component-wise such that

$$\sigma : (s_1, \dots, s_M) \mapsto (\sigma(s_1), \dots, \sigma(s_M))$$

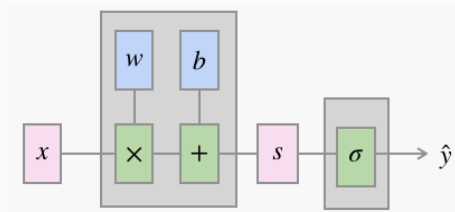
Hence, multiple outputs are computed by

$$\hat{y} = \sigma(w \cdot x + b)$$

Extending ' $\times$ ' to matrix-vector products, the figure remains similar to that of a single perceptron



We explicitly incorporate an auxiliary variable s in the figure to differentiate between the linear and non-linear operations

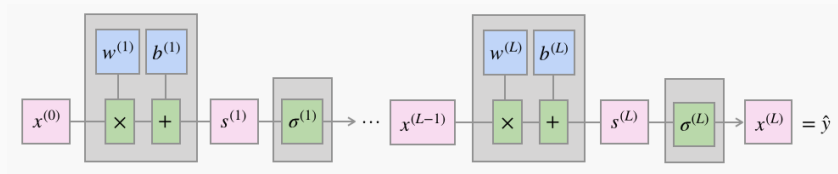


The output of the layer is now computed by

$$s = w \cdot x + b$$

$$\hat{y} = \sigma(s)$$

A **Multi-layer Perceptron (MLP)** concatenates several (stacked) perceptrons such that the output of one 'layer' is the input to the next:



with $x^{(0)} = x$ and $\forall \ell = 1, \dots, L$:

$$s^{(\ell)} = w^{(\ell)} x^{(\ell-1)} + b^{(\ell)}$$

$$x^{(\ell)} = \sigma^{(\ell)}(s^{(\ell)})$$

and $\hat{y} = f(x; w, b) = x^{(L)}$

Instances are processed one at a time. We focus on the n -th instance and obtain a sequence

$$x^{(0)} = x \xrightarrow{w^{(1)}, b^{(1)}} s^{(1)} \xrightarrow{\sigma} x^{(1)} \xrightarrow{w^{(2)}, b^{(2)}} s^{(2)} \xrightarrow{\sigma} \dots \xrightarrow{w^{(L)}, b^{(L)}} s^{(L)} \xrightarrow{\sigma} x^{(L)}$$

with $x^{(0)} = x_n$ and $\forall \ell = 1, \dots, L$

$$s^{(\ell)} = w^{(\ell)} x^{(\ell-1)} + b^{(\ell)} \quad \text{and} \quad x^{(\ell)} = \sigma(s^{(\ell)})$$

The output of the network is then given by $f(x_n; w, b) = \hat{y}_n = x^{(L)}$.

(Remember to cache the $s^{(\ell)}$ and $x^{(\ell)}$ as we will need them for the backward pass)

This is the **forward pass**.

Train an MLP by minimizing a loss over the training set

$$\hat{R}(w, b) = \sum_{n=1}^N V(f(x_n; w, b), y_n)$$

To apply gradient descent, we need to compute the gradient of the loss with respect to the parameters (**batch**):

$$\frac{\partial E}{\partial w_{i,j}^{(\ell)}} \quad \text{and} \quad \frac{\partial E}{\partial b_i^{(\ell)}}$$

We focus on the n -th instance with error $E_n = V(f(x_n; w, b), y_n)$ and aim to compute (**online**)

$$\frac{\partial E_n}{\partial w_{i,j}^{(\ell)}} \quad \text{and} \quad \frac{\partial E_n}{\partial b_i^{(\ell)}}$$

The core principle of back-propagation is the **chain rule** from differential calculus. Let f, g be functions and $(g \circ f)$ their composition, then

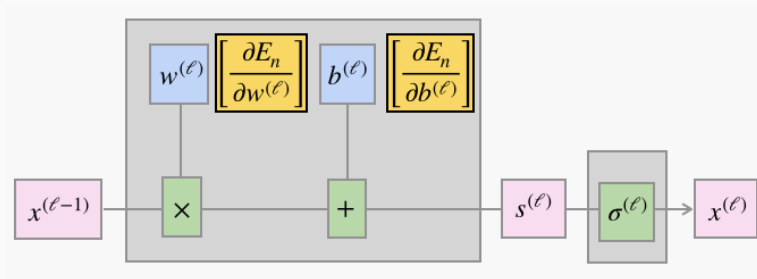
$$(g \circ f)' = (g' \circ f)f'$$

We use the Leibniz notation: Let $z = g(y)$ and $y = f(x)$ and write

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} = g'(y)f'(x) = g'(f(x))f'(x)$$

Let's focus on a single training instance x_n and layer ℓ .

The goal is to compute $\partial E_n / \partial w^{(\ell)}$ and $\partial E_n / \partial b^{(\ell)}$ so that parameters can be updated with gradient descent



We begin by relating the error E_n to the outputs $x^{(L)}$ of the network by computing

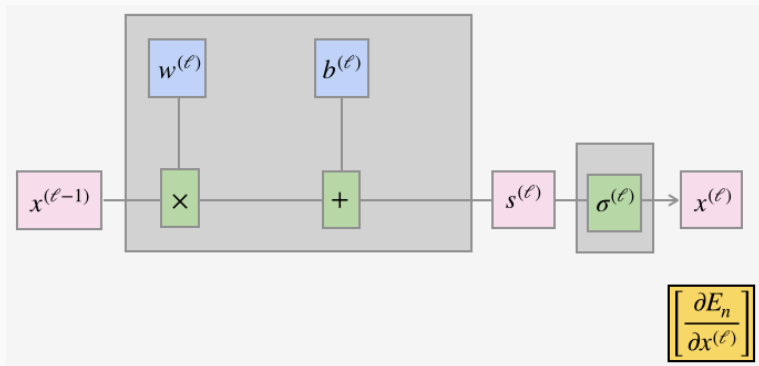
$$\frac{\partial E_n}{\partial x^{(L)}}$$

The quantity depends on the actual loss V . We will come back to this later

Backpropagation is an algorithm to propagate $\frac{\partial E_n}{\partial x^{(L)}}$ backwards through the entire network.

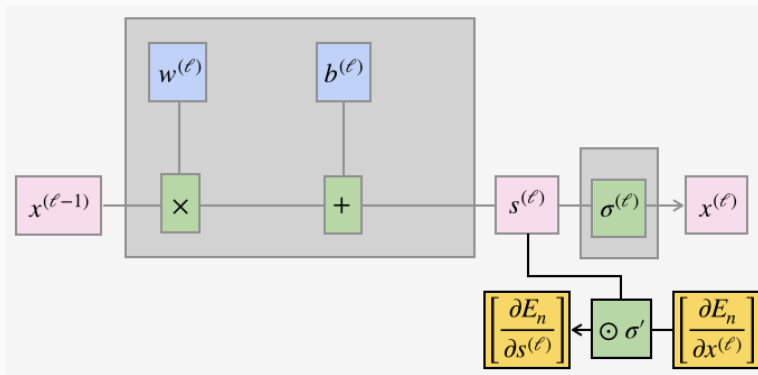
On the way, we compute $\frac{\partial E_n}{\partial x^{(\ell)}}$ and $\frac{\partial E_n}{\partial s^{(L)}}$ for all layers, starting with layer L and ending in layer $\ell = 1$

So, we can assume we either know $\frac{\partial E_n}{\partial x^{(\ell)}}$ or $\ell = L$



We have $x^{(\ell)} = \sigma^{(\ell)}(s^{(\ell)})$; hence, $s^{(\ell)}$ influences E_n only through $x^{(\ell)}$. Let $\odot$ denotes component-wise multiplication, the chain-rule gives:

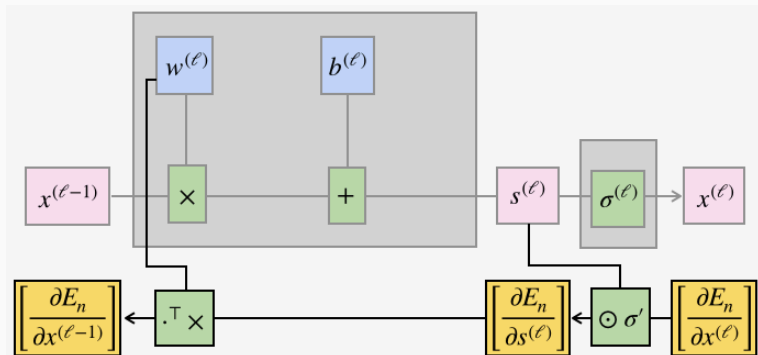
$$\frac{\partial E_n}{\partial s^{(\ell)}} = \frac{\partial E_n}{\partial x^{(\ell)}} \odot \frac{\partial x^{(\ell)}}{\partial s^{(\ell)}} = \frac{\partial E_n}{\partial x^{(\ell)}} \odot \sigma' \left(s^{(\ell)} \right)$$



Since $s^{(\ell)}$ is known from the forward pass, we can now propagate the error from E_n to $s^{(\ell)}$

Similarly, $x^{(\ell-1)}$ influences E_n only through $s^{(\ell)} = w^{(\ell)} \cdot x^{(\ell-1)} + b^{(\ell)}$ and we obtain

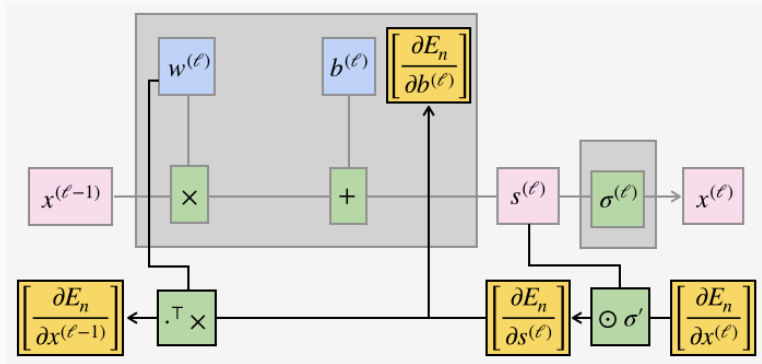
$$\frac{\partial E_n}{\partial x^{(\ell-1)}} = \frac{\partial E}{\partial s^{(\ell)}} \frac{\partial s^{(\ell)}}{\partial x^{(\ell-1)}} = w^{(\ell)\top} \frac{\partial E}{\partial s^{(\ell)}}$$



$\frac{\partial E_n}{\partial x^{(\ell-1)}}$ is the input for propagating the error through layer $\ell - 1$

We have $s^{(\ell)} = w^{(\ell)} \cdot x^{(\ell-1)} + b^{(\ell)}$, and $b^{(\ell)}$ influences layer ℓ only through $s^{(\ell)}$ and we get

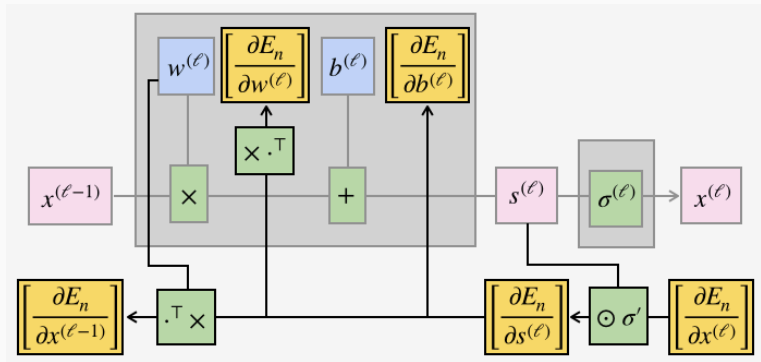
$$\frac{\partial E_n}{\partial b^{(\ell)}} = \frac{\partial E_n}{\partial s^{(\ell)}} \frac{\partial s^{(\ell)}}{\partial b^{(\ell)}} = \frac{\partial E_n}{\partial s^{(\ell)}}$$



The desired quantity is identical to $\frac{\partial E_n}{\partial s^{(\ell)}}$

Similarly, we have $s^{(\ell)} = w^{(\ell)} x^{(\ell-1)} + b^{(\ell)}$ so $w^{(\ell)}$ also influences layer ℓ only through $s^{(\ell)}$, thus

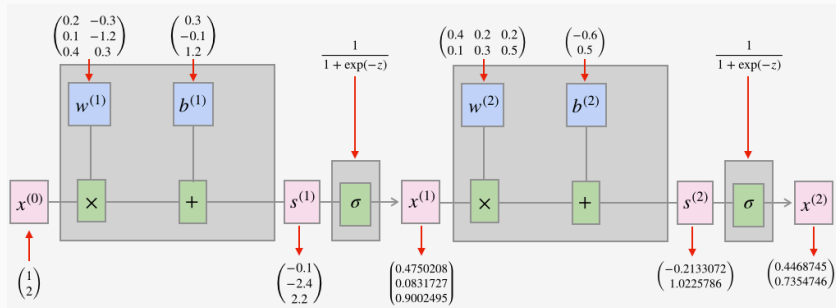
$$\frac{\partial E_n}{\partial w^{(\ell)}} = \frac{\partial E_n}{\partial s^{(\ell)}} \frac{\partial s^{(\ell)}}{\partial w^{(\ell)}} = \frac{\partial E_n}{\partial s^{(\ell)}} \left(x^{(\ell-1)} \right)^\top$$



Since we know $x^{(\ell-1)}$ from the forward pass, we can propagate the error back to the weights $w^{(\ell)}, b^{(\ell)}$

Example

Given an input $x = (1, 2)$ and a desired output $y = (0.01, 0.99)$, a 2-layer network with two stacked units in each layer and sigmoid activation functions is defined as follows:



The **forward pass** computes the output

$$x^{(2)} = \hat{y} = (0.4468745, 0.7354746)^T$$

Cheat Sheet

Forward pass:

Compute the activations

$$x^{(0)} = x_n, \quad s^{(\ell)} = w^{(\ell)} x^{(\ell-1)} + b^{(\ell)} \quad \text{and} \quad x^{(\ell)} = \sigma(s^{(\ell)})$$

Backward pass:

Compute the derivatives of the error wrt the activations

$$\begin{aligned} \left[\frac{\partial E_n}{\partial x^{(L)}} \right] &= \frac{\partial E_n}{\partial x^{(L)}}, & \left[\frac{\partial E_n}{\partial s^{(\ell)}} \right] &= \left[\frac{\partial E_n}{\partial x^{(\ell)}} \right] \odot \sigma'(s^{(\ell)}) \\ \left[\frac{\partial E_n}{\partial x^{(\ell)}} \right] &= (w^{(\ell+1)})^\top \left[\frac{\partial E_n}{\partial s^{(\ell+1)}} \right] \end{aligned}$$

Compute the derivatives of the loss wrt the parameters

$$\left[\frac{\partial E_n}{\partial w^{(\ell)}} \right] = \left[\frac{\partial E_n}{\partial s^{(\ell)}} \right] (x^{(\ell-1)})^\top, \quad \left[\frac{\partial E_n}{\partial b^{(\ell)}} \right] = \left[\frac{\partial E_n}{\partial s^{(\ell)}} \right]$$

Gradient step:

$$w^{(\ell)} \leftarrow w^{(\ell)} - \eta \left[\frac{\partial E_n}{\partial w^{(\ell)}} \right], \quad b^{(\ell)} \leftarrow b^{(\ell)} - \eta \left[\frac{\partial E_n}{\partial b^{(\ell)}} \right]$$